实验3 公钥密码应用

姓名	年级专业	课程名称	实验时间
姚嫣如	23 网络空间安全	网络安全技术	2026.4.17

一、实验目的

1. **深入理解公钥密码学原理**：

   通过编程实践，掌握 Diffie-Hellman (X25519) 密钥交换协议的数学逻辑及工作流程。

2. **掌握身份认证机制**：

   理解中间人攻击（MITM）的原理，并学习如何利用数字签名（Ed25519）确保公钥交换过程中的真实性与完整性。

3. **实践混合加密体系**：

   理解为何在实际应用中采用“非对称加密协商密钥 + 对称加密传输数据”的混合模式（兼顾安全性与效率）。

4. **提升网络安全开发能力**：

   掌握基于 Socket 的网络编程，学习如何在不安全的信道（网络）上构建安全的通信链路。

二、实验要求

1. **编程语言与库**：建议使用 Python 语言，结合 socket 标准库实现网络通信，使用 cryptography 第三方库（特别是 hazmat 原语层）实现加密算法。亦可使用其他编程语言及其对应的密码学库。
2. **环境配置**：实验需在两台机器间进行，通常为**物理机（Host）与虚拟机（VM）**。双方需通过网络协议（TCP/IP）建立连接。
3. **核心算法要求**：
   - **密钥交换**：必须实现 Diffie-Hellman 协议（推荐使用 X25519）。
   - **身份认证**：在密钥协商阶段必须引入数字签名机制（推荐使用 Ed25519）以防御中间人攻击。
   - **对称加密**：使用协商后的共享密钥对文件进行对称加密传输（如 AES 或 Fernet）。
4. **功能实现**：程序应能完整演示“握手协商密钥 -> 身份验证 -> 加密文件发送 -> 接收解密”的全过程。

三、实验环境

- 操作系统：物理机（Windows） + 虚拟机（Ubuntu）
- 开发语言： Python 3.13
- 依赖库： cryptography

四、实验步骤

（一）Diffie-Hellman 密钥交换协议

1. **DH协议**

① 核心意义

允许通信双方在完全不安全的信道（可能被监听）上，协商出一个仅有双方知晓的共享密钥。该密钥随后可用于对称加密算法（如 AES），从而实现端到端的数据保密通信。

② 数学原理

建立在有限域上**离散对数问题 (DLP)** 的计算困难性之上。

- Alice 生成私钥 a，计算并发送公钥

$$A = g^a \ mod \ p$$

Bob 生成私钥 b，计算并发送公钥

$$B = g^b \ mod \ p$$

- Alice 收到 B 后计算：

$$S = B^a \ mod \ p$$

Bob 收到 A 后计算：

$$S = A^b \ mod \ p$$

③ **X25519(ECDH)**

基于**椭圆曲线 (Elliptic Curve Diffie-Hellman, ECDH)** 的现代密钥交换算法。相比经典 DH，X25519 使用更短的密钥长度（**256 位**）即可达到极高的安全性等级（相当于 RSA 3072 位），且计算速度更快，抗侧信道攻击能力强。

将经典 DH 的模幂运算替换为椭圆曲线上的**标量乘法**运算。

2. **实现逻辑**

① 生成身份与初始化

- **私钥生成**：调用 `x25519.X25519PrivateKey.generate()`。这本质上是在本地生成一个巨大的随机数。
- **公钥导出**：从私钥导出公钥

② 建立网络通信

- **Server 端**：监听端口，等待连接。
- **Client 端**：连接 Server。
- **公钥序列化**：socket 传输的是 bytes（字节流），所以需要用 public_bytes() 把公钥对象序列化成二进制。

③ 非对称交换

通过 TCP Socket 建立全双工通道，Client 与 Server 互换公钥字节流。

④ 计算共享密钥

双方各自运行算法，得出相同的 Shared Secret（共享密钥）。

```
shared_key = alice_private_key.exchange(bob_public_key)
```

3. **代码实现**

服务端代码如下：

```python
import socket
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import x25519

def start_dh_server():
    # 1. 生成服务端自己的密钥对
    # 生成私钥
    server_private_key = x25519.X25519PrivateKey.generate()
    # 导出公钥
    server_public_key = server_private_key.public_key()

    # 将公钥对象转换为“字节流”
    server_pub_bytes = server_public_key.public_bytes(
        encoding=serialization.Encoding.Raw,
        format=serialization.PublicFormat.Raw
    )

    #  2. 设置网络连接 (Socket)
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # 绑定 0.0.0.0 表示监听虚拟机所有的网卡，端口选一个没被占用的
    s.bind(('0.0.0.0', 65432))
```

```
    s.listen(1)
    print("等待物理机(客户端)连接...")

    conn, addr = s.accept()
    with conn:
        print(f"连接成功！来自: {addr}")

        # 3. 交换公钥
        # 接收客户端发来的公钥 (X25519 公钥固定 32 字节)
        client_pub_bytes = conn.recv(32)
        # 将接收到的字节转回公钥对象
        client_public_key = x25519.X25519PublicKey.from_public_bytes(client_pub_bytes)

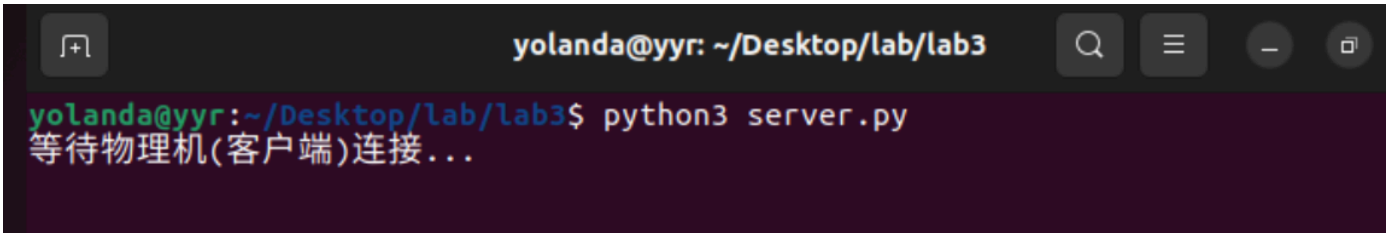
        # 发送服务端的公钥给客户端
        conn.sendall(server_pub_bytes)

        #  4. 计算共享密钥
        # 核心步骤：使用"自己的私钥" + "对方的公钥"
        shared_key = server_private_key.exchange(client_public_key)

        print("\n[结果] 密钥协商成功！")
        print(f"协商出的共享密钥 (Hex): {shared_key.hex()}")

if __name__ == "__main__":
    start_dh_server()
```

先在虚拟机中运行服务端：



客户端代码如下：

```
import socket
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import x25519


def start_dh_client(server_ip):
    #  1. 生成客户端自己的密钥对
    client_private_key = x25519.X25519PrivateKey.generate()
    client_public_key = client_private_key.public_key()
```

```
    client_pub_bytes = client_public_key.public_bytes(
        encoding=serialization.Encoding.Raw,
        format=serialization.PublicFormat.Raw
    )

    # 2. 连接服务端
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        s.connect((server_ip, 65432))
        print(f"成功连接到服务端: {server_ip}")

        # 3. 交换公钥
        # 先发送自己的公钥
        s.sendall(client_pub_bytes)

        # 接收服务端的公钥
        server_pub_bytes = s.recv(32)
        server_public_key = x25519.X25519PublicKey.from_public_bytes(server_pub_bytes)

        # 4. 计算共享密钥
        # 核心步骤：使用"自己的私钥" + "对方的公钥"
        shared_key = client_private_key.exchange(server_public_key)

        print("\n[结果] 密钥协商成功！")
        print(f"协商出的共享密钥 (Hex): {shared_key.hex()}")

    except Exception as e:
        print(f"连接失败: {e}")
    finally:
        s.close()


if __name__ == "__main__":
    target_ip = '192.168.126.128'
    start_dh_client(target_ip)
```

运行客户端，结果如下：

```
"D:\yyr\study\aa 专业课\网络安全技术\.venv\Scripts\python.exe" "D:\yyr\study\aa 专业课\网络安全
成功连接到服务端: 192.168.126.128

[结果] 密钥协商成功!
协商出的共享密钥 (Hex): e6cca2ad6fc703d82987134f8945a0fb02092271c89c8f204047fd6bb0043d18

进程已结束，退出代码为 0
```

```
yolanda@yyr:~/Desktop/lab/lab3$ python3 server.py
等待物理机(客户端)连接...
连接成功！来自：('192.168.126.1', 59822)

[结果]密钥协商成功！
协商出的共享密钥 (Hex): e6cca2ad6fc703d82987134f8945a0fb02092271c89c8f204047fd6bb0043
d18
```

两边密钥完全一致

（二）防御中间人攻击与文件加密传输

1. **中间人攻击（Man-in-the-Middle Attack，MITM）**

   攻击者非法介入通信双方的链路，通过**拦截并伪造公钥**，使得通信双方在不知情的情况下分别与攻击者建立密钥协商。由于原生的 Diffie-Hellman 协议不提供身份认证功能，无法识别公钥是否被篡改。

   ① 三阶段

   - 拦截 Interception

     攻击者通过 Wi-Fi 劫持（伪造热点）、ARP 欺骗或 DNS 劫持，让受害者的流量经过自己的设备。

   - 解密/篡改 Decryption/Manipulation

     由于攻击者成功介入了密钥协商，他可以解密流量，甚至在里面植入恶意代码。

   - 转发 Forwarding

     将处理过的数据发给真正的接收者，让双方察觉不到异常。

   ② 防御措施

   - 数字签名

     即便攻击者拦截了公钥，但无法伪造签名（因为无法获得私钥），签名校验会失败。

   - 数字证书

     在现实的互联网（HTTPS）中，我们使用 CA 机构颁发的证书来证明“这个公钥确实属于该发送端”。

   - 多因素认证 MFA

     增加一道验证关卡。

2. **实现逻辑**

   ① 数字签名 Ed25519

   - 双方在交换 X25519 公钥前，先用各自的 Ed25519 私钥对公钥进行签名。
   - 接收方收到数据后，**拆分为公钥和签名**，利用预存的合法公钥进行校验。若中间人试图替换公钥，将无法伪造合法的签名，从而被发现。

② 密钥派生 KDF

- 利用 HKDF (HMAC-based Key Derivation Function) 对协商出的共享秘密进行二次处理。
- 确保了生成的密钥具有足够的随机性和标准长度，适合作为 AES 对称加密的 Key。

③ 文件加密传输

- 使用 cryptography.fernet（基于 AES-128 的对称加密方案）对文件内容进行加密。
- 通过已经过身份认证的安全通道发送密文。
- 服务端收到密文后，使用相同的派生密钥进行解密还原

3. **代码实现**

- 生成密钥:

```python
from cryptography.hazmat.primitives.asymmetric import ed25519
from cryptography.hazmat.primitives import serialization

# 1. 生成服务端的身份密钥（私钥和公钥）
s_priv = ed25519.Ed25519PrivateKey.generate()
s_pub = s_priv.public_key()

# 2. 生成客户端的身份密钥（私钥和公钥）
c_priv = ed25519.Ed25519PrivateKey.generate()
c_pub = c_priv.public_key()

# 3. 将这些密钥保存到当前文件夹
with open("server_identity.priv", "wb") as f:
    f.write(s_priv.private_bytes(
        encoding=serialization.Encoding.Raw,
        format=serialization.PrivateFormat.Raw,
        encryption_algorithm=serialization.NoEncryption()
    ))

with open("server_identity.pub", "wb") as f:
    f.write(s_pub.public_bytes(
        encoding=serialization.Encoding.Raw,
        format=serialization.PublicFormat.Raw
    ))

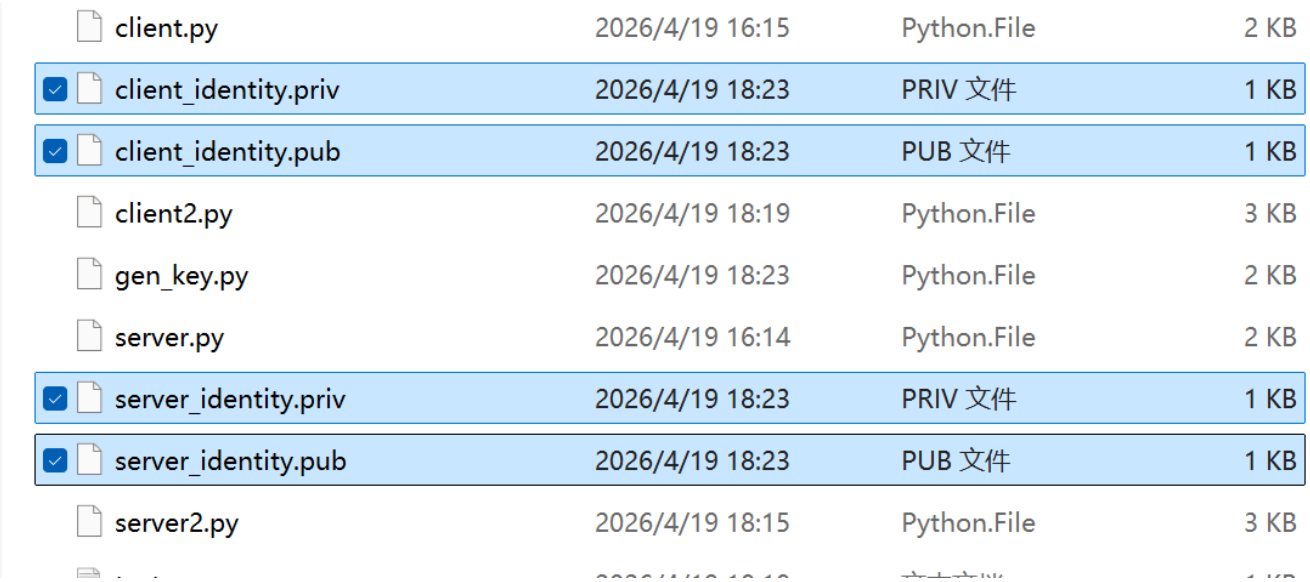
with open("client_identity.priv", "wb") as f:
    f.write(c_priv.private_bytes(
        encoding=serialization.Encoding.Raw,
        format=serialization.PrivateFormat.Raw,
        encryption_algorithm=serialization.NoEncryption()
    ))

```

```python
with open("client_identity.pub", "wb") as f:
    f.write(c_pub.public_bytes(
        encoding=serialization.Encoding.Raw,
        format=serialization.PublicFormat.Raw
    ))

print("身份密钥文件已成功生成！")
```

文件夹中出现如下四个文件：



client.py	2026/4/19 16:15	Python.File	2 KB
client_identity.priv	2026/4/19 18:23	PRIV 文件	1 KB
client_identity.pub	2026/4/19 18:23	PUB 文件	1 KB
client2.py	2026/4/19 18:19	Python.File	3 KB
gen_key.py	2026/4/19 18:23	Python.File	2 KB
server.py	2026/4/19 16:14	Python.File	2 KB
server_identity.priv	2026/4/19 18:23	PRIV 文件	1 KB
server_identity.pub	2026/4/19 18:23	PUB 文件	1 KB
server2.py	2026/4/19 18:15	Python.File	3 KB

- 通过scp将四个文件传输至虚拟机中：

  在虚拟机上安装 SSH

```
yolanda@yyr:~/Desktop/lab/lab3$ sudo apt install openssh-server
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openssh-server is already the newest version (1:8.9p1-3ubuntu0.14).
0 upgraded, 0 newly installed, 0 to remove and 570 not upgraded.
```

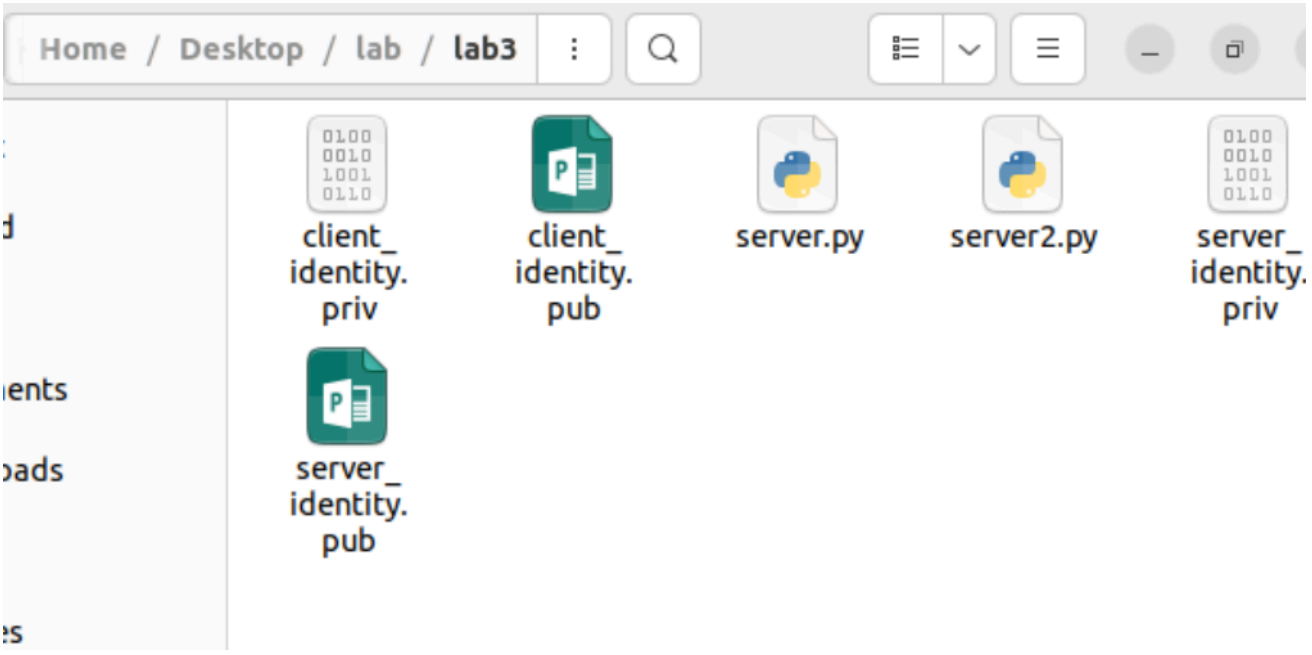
  启动 ssh 服务：

```
yolanda@yyr:~/Desktop/lab/lab3$ sudo systemctl start ssh
yolanda@yyr:~/Desktop/lab/lab3$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enable>
     Active: active (running) since Fri 2026-04-17 14:02:53 CST; 2 days ago
       Docs: man:sshd(8)
             man:sshd_config(5)
   Main PID: 1007 (sshd)
      Tasks: 1 (limit: 4575)
     Memory: 5.5M
        CPU: 424ms
     CGroup: /system.slice/ssh.service
             └─1007 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

使用SCP命令将文件传输至虚拟机：

_identity. 匹配所有以_identity 命名结尾的文件

```
PS D:\yyr\study\aa 专业课\网络安全技术\lab3> scp *_identity.* yolanda@lab:/home/yo
landa/Desktop/lab/lab3
yolanda@lab's password:
client_identity.priv                          100%   32    10.4KB/s   00:00
client_identity.pub                           100%   32    15.6KB/s   00:00
server_identity.priv                          100%   32    15.6KB/s   00:00
server_identity.pub                           100%   32    31.2KB/s   00:00
```



- 服务端代码如下：

```python
import socket
import base64
from cryptography.hazmat.primitives.asymmetric import x25519, ed25519
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.fernet import Fernet

def start_secure_server():
    # 1. 加载身份密钥 (用于签名和验签)
    with open("server_identity.priv", "rb") as f:
        my_sign_priv = ed25519.Ed25519PrivateKey.from_private_bytes(f.read())
    with open("client_identity.pub", "rb") as f:
        peer_sign_pub = ed25519.Ed25519PublicKey.from_public_bytes(f.read())

    # 2. 准备 DH 密钥交换
    my_dh_priv = x25519.X25519PrivateKey.generate()
```

```
    my_dh_pub_bytes = my_dh_priv.public_key().public_bytes(0, 0) # Raw 格式

    # 核心：对自己的 DH 公钥进行数字签名
    signature = my_sign_priv.sign(my_dh_pub_bytes)

    # 3. Socket 连接
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind(('0.0.0.0', 65432))
    s.listen(1)
    print("安全服务端已启动，等待连接...")
    conn, addr = s.accept()

    with conn:
        # 4. 接收客户端的 [公钥(32字节) + 签名(64字节)]
        data = conn.recv(96)
        client_dh_pub_bytes = data[:32]
        client_sig = data[32:]

        # 验证客户端签名
        try:
            peer_sign_pub.verify(client_sig, client_dh_pub_bytes)
            print("身份验证成功：公钥确实来自客户端！")
        except:
            print("警告：身份验证失败！可能是中间人攻击！")
            return

        # 5. 发送自己的 [公钥 + 签名]
        conn.sendall(my_dh_pub_bytes + signature)

        # 6. 计算共享密钥并使用 HKDF 派生对称密钥
        shared_secret = my_dh_priv.exchange(x25519.X25519PublicKey.from_public_bytes(client_dh_pub_bytes))

        # HKDF 转换 (将 DH 结果变成 32 字节的 AES 密钥)
        derived_key = HKDF(algorithm=hashes.SHA256(), length=32, salt=None, info=b'file-transfer').derive(shared_secret)
        # Fernet 库需要 base64 格式的密钥
        fernet_key = base64.urlsafe_b64encode(derived_key)
        cipher = Fernet(fernet_key)

        # 7. 接收加密的文件内容
        print("等待接收加密文件...")
        encrypted_data = conn.recv(4096)
        decrypted_data = cipher.decrypt(encrypted_data)

```

```
        with open("received_file.txt", "wb") as f:
            f.write(decrypted_data)
        print("文件接收成功并已解密为 'received_file.txt'")

if __name__ == "__main__":
    start_secure_server()
```

先运行服务端：

```
yolanda@yyr:~/Desktop/lab/lab3$ python3 server2.py
安全服务端已启动，等待连接...
```

- 客户端代码如下：

```
import socket
import base64
from cryptography.hazmat.primitives.asymmetric import x25519, ed25519
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives import serialization


def start_secure_client(server_ip):
    # 1. 加载身份密钥
    with open("client_identity.priv", "rb") as f:
        my_sign_priv = ed25519.Ed25519PrivateKey.from_private_bytes(f.read())
    with open("server_identity.pub", "rb") as f:
        peer_sign_pub = ed25519.Ed25519PublicKey.from_public_bytes(f.read())

    # 2. 准备 DH
    my_dh_priv = x25519.X25519PrivateKey.generate()
    my_dh_pub_bytes = my_dh_priv.public_key().public_bytes(
        encoding=serialization.Encoding.Raw,
        format=serialization.PublicFormat.Raw
    )
    signature = my_sign_priv.sign(my_dh_pub_bytes)

    # 3. 连接
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((server_ip, 65432))

    # 4. 发送 [公钥 + 签名]
    s.sendall(my_dh_pub_bytes + signature)
```

```
31
32      # 5. 接收服务端的 [公钥 + 签名]
33      data = s.recv(96)
34      server_dh_pub_bytes = data[:32]
35      server_sig = data[32:]
36
37      # 6. 验证服务端签名
38      try:
39          peer_sign_pub.verify(server_sig, server_dh_pub_bytes)
40          print("身份验证成功：公钥确实来自服务端！")
41      except:
42          print("异常：服务端签名错误！")
43          return
44
45      # 7. 派生对称密钥
46      shared_secret =
    my_dh_priv.exchange(x25519.X25519PublicKey.from_public_bytes(server_d
    h_pub_bytes))
47      derived_key = HKDF(algorithm=hashes.SHA256(), length=32,
    salt=None, info=b'file-transfer').derive(shared_secret)
48      cipher = Fernet(base64.urlsafe_b64encode(derived_key))
49
50      # 8. 读取文件、加密并发送
51      file_to_send = "test.txt"  # 请确保本地有一个 test.txt
52      with open(file_to_send, "rb") as f:
53          content = f.read()
54
55      encrypted_content = cipher.encrypt(content)
56      s.sendall(encrypted_content)
57      print(f"文件 {file_to_send} 已加密发送。")
58      s.close()
59
60
61  if __name__ == "__main__":
62      ip = '192.168.126.128'
63      start_secure_client(ip)
```

```
"D:\yyr\study\aa 专业课\网络安全技术\.venv\Scripts\python.exe"
身份验证成功：公钥确实来自服务端！
文件 test.txt 已加密发送。

进程已结束，退出代码为 0
```

```
yolanda@yyr:~/Desktop/lab/lab3$ python3 server2.py
安全服务端已启动，等待连接...
身份验证成功：公钥确实来自客户端！
等待接收加密文件...
文件接收成功并已解密为 'received_file.txt'
```

```
yolanda@yyr:~/Desktop/lab/lab3$ ls
client_identity.priv  received_file.txt  server_identity.priv  server.py
client_identity.pub   server2.py         server_identity.pub
yolanda@yyr:~/Desktop/lab/lab3$ cat received_file.txt
Hello! This is a test file.yolanda@yyr:~/Desktop/lab/lab3$
```

五、体会与心得

通过本次实验，我将课堂上抽象的 Diffie-Hellman (X25519) 数学原理转化为了实际可运行的代码。我深刻体会到，虽然密钥交换在理论上是完美的，但在实际网络通信中，如何处理字节流、序列化公钥以及建立稳定的 Socket 连接，都是实现安全通信的关键环节。

在作业 1 中，虽然成功协商了密钥，但这种方式无法抵御中间人攻击。作业 2 引入 Ed25519 数字签名的过程让我明白：在密码学中，“你是谁”和“信息加密”同样重要。没有身份验证的加密就像是一个没有封条的保险箱，虽然内部安全，但可能被掉包。

实验中通过“非对称加密（协商密钥）+ 对称加密（传输文件）”的组合，让我理解了现代安全协议（如 HTTPS/TLS）的设计逻辑。这种模式既利用了公钥密码学无需预共享密钥的便利，又发挥了对称加密在处理大数据时的高效率，是安全性与性能的平衡艺术。